When you want RSpec to be strict about your verifying doubles, just pass -rsupport/verify_doubled_constants on the command line:

```
$ rspec -rtrail_map -rsupport/verify_doubled_constants
```

Your specs will correctly fail, and RSpec will warn you that the class name doesn't exist. If you use this approach, we recommend that you develop with this setting *off*, but configure your continuous integration (CI) server to run with the setting *on*.

Make It Easy to Replicate Your CI Setup



Repeatability is important when you're setting up a CI system. Few things are more frustrating than a spec passing on your local machine but failing on the CI server.

If you're going to use certain options only with CI, such as the verify_doubled_constant_names setting, we recommend putting all of these options into a script or Rake task you can run locally. That way, when a spec fails on CI, you can just run something like ./script/ci_build and diagnose the issue on your machine.

We'll talk more about integration with Rake in Appendix 1, *RSpec and the Wider Ruby Ecosystem*, on page 293.

Wrapping Up

As we wrap up, let's look at the trade-offs we've seen. Verifying doubles do the following things:

- They raise errors when your code calls a dependency incorrectly.
- They can only do so when the dependency actually exists.
- They revert silently to regular doubles if the dependency doesn't exist.

To deal with that last item, you can create your doubles from Ruby class names instead of strings. It's only practical to do so if you've already written code for the dependency, and if it's not too expensive to load it. If you can't use a Ruby class, you can still double-check your constant names by setting verify_doubled_constant_names when you run your whole suite.

Using verifying doubles correctly takes a little extra up-front care. But the benefits to your project are well worth it.